

AI와 함께 일하는 법

Claude Code

설치부터 포트폴리오 배포까지, 초보자 완전 정복

Claude Code

bkit Plugin

VS Code

Vercel

Claude Code, 왜 쓰는가?

Claude 제품군 — 용도에 맞게 골라 쓴다

Claude

가볍게 대화하며 바로 처리

회의 정리

아이디어 정리

Cowork

폴더를 지정해 같이 들여다보며 작업

리서치

사업계획

Claude Code ★

직접 만들고 구축까지 끝낸다

포트폴리오

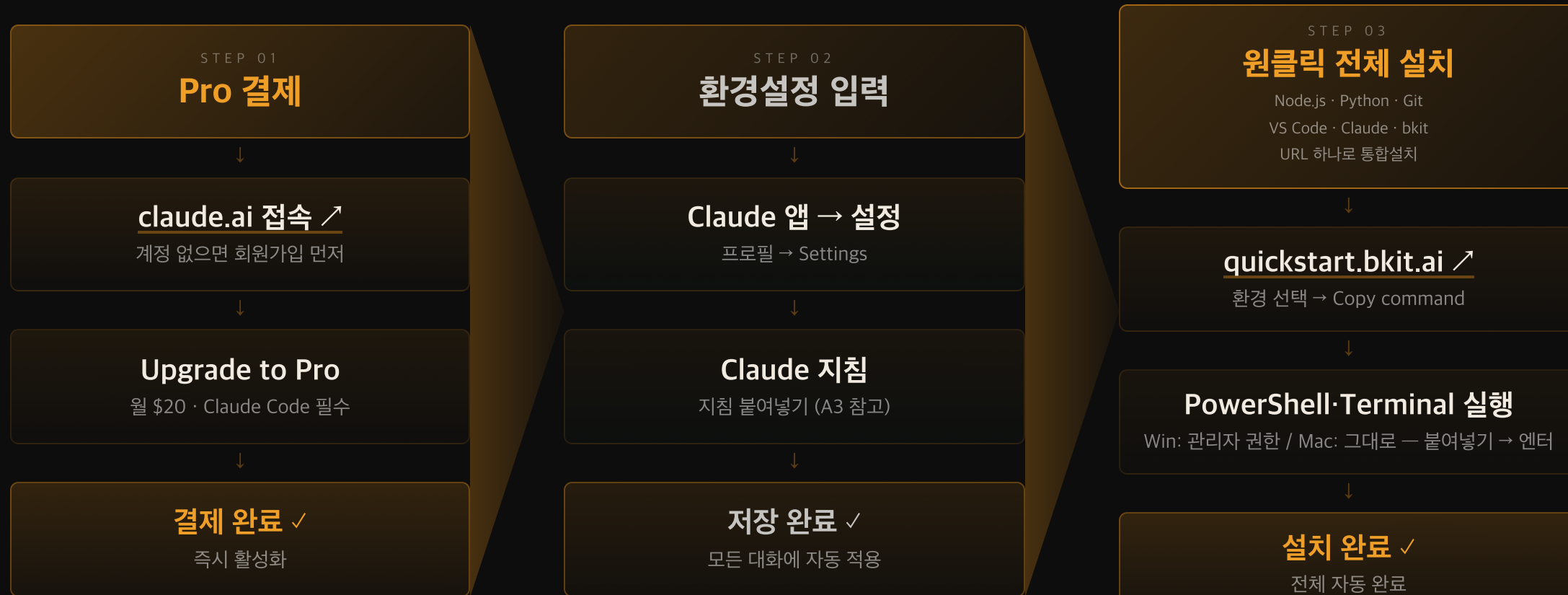
웹사이트

자동화

TIP 이 강의는 전부 Claude Code 기준으로 진행 — Claude·Cowork는 비교용으로만 알아두면 충분

Claude Code, 준비하기

시작 전 이 3가지를 순서대로 준비한다



TIP 다른 YouTube·가이드는 위 과정을 하나씩 해야 해서 복잡함 — 위 절차로 "원클릭" 끝

VS Code와 연결하기

폴더 지정 → Claude 호출 — 이 두 단계가 시작의 전부다

STEP 01

별도 폴더 생성

기존 폴더 X
새 폴더 권장

STEP 02

VS Code에서 열기

작업 파일·이미지
폴더에 정리

STEP 03

Ctrl + Shift + P

Mac: Cmd + Shift + P

명령 팔레트 열기

STEP 04

FOCUS INPUT

명령창에

FOCUS INPUT 입력

Claude Code 입력창

활성화

⚠ **폴더 지정 시 주의사항** Claude는 지정된 폴더 내 파일을 수정·삭제 할 수 있습니다. 반드시 **별도의 새 폴더** 를 만들어 지정하세요 — 중요한 기존 파일과 혼용 금지.

bkit이 뭔데?

Claude만 쓸 때와 bkit을 더했을 때, 결과물 완성도가 달라진다

나 혼자 + Claude

지시 → 코딩 → 오류 → 반복
방향이 흔들리고
중간에 길을 잃기 쉬움

VS

나 + bkit + Claude

16명의 AI 에이전트가 기획부터 QA까지
전 과정을 체계적으로 이끌어 줌

기획

UIUX

프론트

백엔드

QA

DB

보안

문서화

핵심 bkit은 Claude를 제어하는 AI 에이전트 팀 — 임의 코딩·반복 작업 문제를 크게 줄여준다

핵심 명령어 3개만 기억해

bkit Plugin — 이 순서대로만 하면 된다

STEP 01

/plan-plus

기획 단계

아이디어를 구체화하고
방향 정리. 무엇을 만들지
먼저 명확히 한다

STEP 02

/pdca

실행 단계

기획 완료 후
계획 → 코딩 → 검증 절차를
순서대로 이끌어 간다

STEP 03

/gap-detector

검증 단계

개발 완료 후 오류 점검 및 QA.
빠진 부분 없는지
최종 확인

TIP 명령어 전에 작업 내용을 정리한 .md 파일을 폴더에 미리 넣어두면 훨씬 정확하게 작동해요

전체 작업 흐름 한눈에

폴더 지정부터 배포까지 — 5단계로 끝난다

STEP 01

폴더 지정

새 폴더 생성
자료 정리

STEP 02

/plan-plus

기획
방향 확정

STEP 03

/pdca

코딩
실행

STEP 04

/gap-detector

검증
QA

STEP 05

배포

Vercel
원클릭

중간 얼럿 · 선택 메시지

작업 중 묻거나 선택하라는 메시지는 대부분
Yes로 진행해도 큰 문제 없음

이후 자연어로 다듬기

기본 결과물이 나오면 **경로, 캡처 이미지** 등을
구체적으로 지시하며 세부 조정

핵심 막히면 대부분 Yes로 진행해도 괜찮다 — 세부 조정은 결과물이 나온 뒤 자연어로

실전 예시 — 포트폴리오 만들기

단일 HTML · 다크 네이비 · 반응형 · 라이트박스 갤러리

직접 따라하려면 뒤에 나올 실습 1 참고

1

폴더에 포트폴리오 자료 정리

경력 텍스트, 작업물 이미지, 방향 .md 파일 준비

2

/plan-plus 로 기획 확정

섹션 구성, 컬러, 인터랙션 방향을 AI와 함께 구체화

3

/pdca 로 코딩 실행

HERO → ABOUT → CASE → CAREER → GALLERY → CONTACT

4

자연어로 세부 다듬기

스크린샷·경로 지시로 수정 후 /gap-detector QA

5

Vercel 배포

GitHub 연동 후 원클릭 배포 → URL 공유

결과물 섹션 구조

HERO

- 핵심 지표 카운트업
- 마우스 호버 애니메이션

ABOUT

- 역량 핵심 필러 3~4개
- 스크롤 reveal 효과

CASE STUDIES

- 배경 → 실행 → 결과
- PAR 구조 카드

GALLERY

- 썸네일 카드
- 라이트박스 뷰어

TIP 사전에 Gemini·GPT로 방향 정리 후 .md 파일로 넘기면 훨씬 빠름

메모리가 꽉 차기 전에 저장하라

Claude Code 입력창 좌측 하단 원형 게이지 — 80% 이상이면 저장 타이밍

메모리 게이지 — 입력창 좌측 하단 원형 아이콘

85%

△ 80% 이상 — 지금 저장해야 할 타이밍

STEP 01

claude.md 저장

현재 작업 맥락을
claude.md에 저장 요청

STEP 02

새 창 열기

Claude Code 새 창
같은 폴더 지정

STEP 03

이어서 작업

"claude.md 읽고
작업 계속하자"로 재개

TIP 새 창으로 옮겨도 claude.md만 읽히면 맥락은 그대로 이어진다

지금까지 배운 것 중 **마무리에 챙길 4가지**

새로운 내용이 아니라 — 앞에서 다룬 것 중 다시 한번 짚어볼 것들

.md로 지시하고, 저장하라

AI는 .md를 가장 잘 읽는다. 지시사항도 .md로 정리하고, 메모리 찰 땐
claude.md로 저장 후 새 창에서 이어가기

[.md 권장](#)

환경설정 지침을 꼭 입력하라

Claude 앱 개인 환경설정에 가이드라인을 입력해두면 임의 코딩·흐름 끊김을
방지할 수 있다. 샘플은 Appendix 참고

[설정 필수](#)

배포는 Vercel

포트폴리오·웹사이트 배포가 필요하다면 Vercel 가입 추천. GitHub 연동 후
원클릭 배포 가능하며 무료 플랜으로도 충분하다

[Vercel 바로가기 ↗](#)

GitHub 백업

작업 중 파일이 삭제되거나 망가질 수 있다. GitHub 주기적 백업 권장.

[GitHub 바로가기 ↗](#)

마지막으로 한 번 더 이 4가지만 챙기면 오늘 배운 건 다 써먹은 거다

이론은 끝났다 — 지금 바로 시작하기

오늘 배운 걸로 손으로 직접 해볼 시간

지금 할 일 딱 하나

새 폴더 하나 만들고 /plan-plus 입력해보기

더 깊이 알고 싶다면 — 참고 자료

- 포트폴리오는 **실습 1**, 프레젠테이션 자료는 **실습 2** — 바로 다음 페이지부터
- 환경설정 지침 전문은 **Appendix A3**에서 복사해서 붙여넣기
- 비용이 걱정되면 **Appendix A7** 확인

핵심 완벽한 계획보다 첫 폴더를 만드는 게 먼저다 — 나머지는 Claude Code가 같이 풀어간다

HANDS-ON

실습하기

포트폴리오/프리젠테이션 만들기

실습 1 — 포트폴리오 만들기

참고 파일 :

[Sample_PORTFOLIO_TEMPLATE_GUIDE.md](#)

[실제 사례 참고하기 ↗](#)

사용 순서 — 초보자도 이대로만 하면 된다

추천 프롬프트 예시 — 그대로 복사해서 쓰세요

1 파일을 폴더에 넣는다

`Sample_PORTFOLIO_TEMPLATE_GUIDE.md` 와 내 경력 자료(이미지, 텍스트)를 같은 폴더에 넣기

2 VS Code에서 해당 폴더 열기

Ctrl+Shift+P (Mac: Cmd+Shift+P) → FOCUS INPUT → Claude Code 입력창 활성화

3 /plan-plus 입력 후 프롬프트 붙여넣기

우측 프롬프트 예시를 그대로 복사해서 입력창에 붙여넣기

4 /pdca 로 실행

기획 완료 후 /pdca 입력 → 코딩 자동 진행. Yes로 계속 진행

5 자연어로 다듬기 → 배포

/gap-detector QA 후 Vercel로 배포

PROMPT 01 — 기획 시작

폴더 안의 `Sample_PORTFOLIO_TEMPLATE_GUIDE.md` 를 읽고, 동일한 구조와 디자인으로 내 포트폴리오를 만들어줘.
내 자료는 폴더 안에 있어. 먼저 어떤 섹션 구성과 컬러 방향으로 만들지 제안해줘.

PROMPT 02 — 내 정보 반영

가이드의 디자인 시스템(CSS 토큰, 섹션 순서)은 그대로 유지하고,
내 경력과 이미지 파일들을 적용해서 단일 HTML 파일로 만들어줘.
이미지는 `capture/g/` 폴더에 있어.

PROMPT 03 — 수정 요청

HERO 섹션의 수치를 [실적 수치]로 바꾸고,
GALLERY 썸네일은 `gal-01.jpg ~ gal-04.jpg` 4장으로 구성해줘.

TIP 프롬프트는 순서대로 — 기획 제안 받기 → 내 정보 반영 → 세부 수정 순으로 진행

실습 2 — 프레젠테이션 자료 만들기

참고 파일 : `slide-template.html` + `slide-template-guide.md`

사용 순서 — 이 강의안과 똑같은 방식

추천 프롬프트 예시 — 그대로 복사해서 쓰세요

1 두 파일을 폴더에 넣는다

`slide-template.html`과 `slide-template-guide.md`를 같은 폴더에 넣기

2 VS Code에서 해당 폴더 열기

Ctrl+Shift+P (Mac: Cmd+Shift+P) → FOCUS INPUT → Claude Code 입력창 활성화

3 우측 프롬프트를 그대로 붙여넣기

발표 주제와 슬라이드별로 하고 싶은 말을 자유롭게 채워서 전달

4 완성된 HTML을 바로 확인

더블클릭으로 브라우저에서 열어 슬라이드 넘기며 확인, 수정 요청은 자연어로

5 Cmd/Ctrl+P로 PDF 저장

용지 크기는 기본값(Default) 유지 — 슬라이드 1장이 PDF 1페이지가 됨

PROMPT 01 — 기획 시작

첨부한 `slide-template.html`을 템플릿으로 써서 발표자료를 만들어줘.
`slide-template-guide.md`의 규칙을 그대로 따라줘.

PROMPT 02 — 내용 채우기

발표 주제는 [주제]이고, 슬라이드는 [장수]장 정도로 생각해.
슬라이드별로 하고 싶은 말은 다음과 같아: [슬라이드별 내용을 자유롭게 적기]

PROMPT 03 — 수정 요청

3번 슬라이드 표 내용을 [새 내용]으로 바꾸고,
전체적으로 글자 크기를 조금 더 키워줘.

TIP CSS·JS는 그대로 두고 본문 슬라이드 블록만 수정/추가/삭제하는 게 원칙

REFERENCE

Appendix

본편에서 다 못한 이야기 — 필요할 때 펼쳐보는 참고 자료

Claude 제품군 상세 비교

어떤 상황에서 무엇을 쓰면 되는지 한눈에

구분	Claude (채팅)	Cowork	Claude Code
사용 방식	웹·앱 채팅	폴더 공유	VS Code 연동
주요 용도	회의 정리, 아이디어, 문서 작성	리서치 리포트, 사업계획, 심층 분석	자동화, 포트폴리오, 웹사이트, 피그마
파일 접근	업로드 방식	폴더 직접 공유	폴더 직접 읽기·쓰기·삭제
난이도	쉬움	보통	다소 어려움
추천 모델	Sonnet	Sonnet / Opus	Sonnet (기본) / Opus (복잡한 작업)
bkit 활용	해당 없음	해당 없음	강력 추천 — bkit 명령어 3종

핵심 결국 대부분의 실전 작업은 Claude Code(VS Code 연동) 한 곳에서 끝난다

Chat은 말, Code는 확인

이 강의안을 직접 고치면서 실제로 겪은 일

Claude Chat에 "고쳐줘" 요청

"하단 푸터 통일, 표·카드 가운데 정렬, 오버플로우 — 모두 수정했습니다 ✅"

→ 실제 파일을 열어보면 15장 중 4장만 반영, 11장은 그대로

코드를 직접 열어 대조할 수 없으니 "됐다"는 답변만 믿을 수밖에 없음

VS

Claude Code + /pdca로 같은 요청

15개 슬라이드를 한 줄씩 직접 열어 동일한 규칙으로 일괄 수정

수정 후 헤드리스 브라우저로 실제 화면을 스크린샷 촬영해 대조

→ A1 표 줄바꿈, A3 헤더 overflow를 스크린샷에서 발견해 그 자리에서 재수정

핵심 "말"이 아니라 "파일을 열어 직접 대조"할 수 있다는 것 — 그게 Claude Code의 가치다

Claude 환경설정 지침

한 번 설정하면 이후 모든 대화에 자동 적용됩니다

[다음 페이지에 지침 전문 이어짐 →](#)



AI 이용 시 자주 겪는 불편 → 지침으로 해결

요청하지 않은 기능을 멋대로 추가함	→	임의 확장 금지 지침
코드 일부를 "// 여기에 구현" 주석으로 날림 처리	→	껍데기 코딩 금지 지침
동의 없이 다른 파일까지 같이 수정해버림	→	독단적 파일 수정 금지
색상·폰트를 제멋대로 인라인으로 덮어씀	→	디자인 토큰 SSOT 원칙
"완벽히 수정했습니다" 말만 하고 확인이 어려움	→	검증 위주 보고 원칙

TIP 처음엔 길어 보여도 한 번 설정하면 끝 — 특히 "확인 후 시작" 원칙만 잘 지켜도 반복 작업이 눈에 띄게 줄어요

환경설정 지침 전문

아래 전체 내용을 복사 → Custom Instructions에 붙여넣기

1. 협업 체계

나는 비개발자로서 기획·방향을 총괄합니다. 내 지시와 AI 아키텍트 설계서를 병렬 수용하여 실무 코드로 구현하세요.

2. 임의 판단 및 과사용 코딩 금지

- 임의 확장 금지: 요청하지 않은 기능 추가 절대 금지.
- 껌데기 코딩 금지: 로직을 주석(// 여기에 구현)으로 날림 처리 금지.
- 독단적 파일 수정 금지: 연관 파일 수정 전 반드시 보고 후 승인.

3. UI/UX 표준

- 색상: 디자인 토큰만 사용, 인라인 하드코딩 금지.
- 접근성: 11px 미만 폰트 금지.
- 클릭 영역: 버튼·링크 최소 44px 확보.

4. 작업 진행 원칙

- 큰 작업 전 "이렇게 진행할게요, 맞나요?" 확인 후 시작.
- 한 번에 하나씩: 잘게 쪼개 단계별 진행.
- 오류 시 전체 재작성 금지 → 오류 메시지 보고 후 부분 수정.

5. 모델 및 세션 관리

- 기본 모델: Sonnet 사용 (복잡한 설계만 Opus).
- 메모리 80% 이상 시: claude.md 저장 후 새 창에서 계속.
- 지시사항은 .md 파일로 폴더에 미리 정리해서 제공.

6. 검증 위주 보고 원칙

완료 시 반드시 아래 3가지 보고:

- ① 수정·신설 파일 경로 목록
- ② 핵심 코드 스니펫 (축약 없이)
- ③ 연관 파일 영향 위험 요인 고지

TIP 6개 섹션 중 "2. 임의 판단 금지"와 "4. 작업 진행 원칙"이 실전에서 가장 체감 효과가 크다

Claude Code를 잘 쓰는 법

본편 마무리 팁 4가지에 더해 — 작업 습관으로 알아두면 좋은 것들

TIP 01

지시는 구체적으로

"이쁘게 만들어줘" (X)
"다크 배경, 골드, 카드 3열" (O)

TIP 02

잘 된 결과물은 .md로 저장

"이전처럼" 보다
"폴더의 .md 참고해서"가 훨씬 정확

TIP 03

막힐 땐 오류 복붙 or 캡처

설명 말고
오류·터미널 메시지 그대로 붙여넣기

TIP 04

한 번에 다 요청하지 마라

/plan-plus → /pdca → /gap-detector
순서가 결국 더 빠름

TIP 05

큰 작업 전 반드시 확인

"이렇게 진행할게요, 맞나요?"를
Claude가 먼저 묻게 지침에 설정

TIP 06

끝나기 전에 끼어들지 말기

새 메시지·창 닫기는 작업 끝난 뒤에
완료 표시 나올 때까지 기다리기

TIP 07

Sonnet으로 충분하다

기본은 Sonnet.
복잡한 아키텍처 설계·추론이 필요할 때만
Opus 전환

TIP 08

작업 전 폴더부터 정리

이미지·텍스트·md 지시서를
새 폴더에 미리 정리하면 훨씬 정확해진다

마지막 한마디 이 8가지만 기억해도 Claude Code 초보 단계는 충분히 넘어선다

GitHub — 안전하게 보관하기

코드와 파일을 클라우드에 저장해두는 서비스 — 컴퓨터가 망가져도 안전하고, 어디서든 로그인만 하면 이어서 작업할 수 있다

1 [github.com](#) 가입

이메일만 있으면 충분

2 저장소(Repository)* 하나 만들기

"New" 버튼 → 이름 정하고 Create (포트폴리오 자료 넣을 "방" 하나 만드는 것)

3 Claude Code에게 요청

"이 폴더를 GitHub에 올려줘" — 커밋(Commit)* 명령어는 외울 필요 없음

흔히 하는 실수

공개(Public)*·비공개(Private)* 구분 안 하기 → **개인정보·API 키 있으면 비공개로**

커밋 없이 며칠치 작업 몰아서 하기 → **중간중간 저장해야 되돌릴 지점이 생김**

터미널 명령어를 직접 외워서 치려다 막힘 → **Claude Code에게 시키면 인증까지 대신 처리**

커밋 메시지 의미 없이 적기 → **"수정함"(X) / "포트폴리오 사진 갤러리 추가"(O)**

* 저장소(Repository) — 파일을 모아두는 공간 · 커밋(Commit) — 저장 시점을 기록하는 것 · 공개/비공개(Public/Private) — 누구나 볼 수 있는지 여부

핵심 명령어를 외울 필요 없다 — Claude Code에게 시키면 저장부터 업로드까지 대신 처리한다

Vercel — 가장 쉬운 배포 방법

만든 HTML을 실제 인터넷 주소(URL)로 띄워주는 서비스 — 포트폴리오는 결국 링크로 보여줘야 한다. GitHub과 연결해두면 커밋할 때마다 자동으로 다시 배포(Deploy)*

1 vercel.com → GitHub로 가입

"Continue with GitHub" — 계정 새로 안 만들어도 됨

2 가져오기(Import)*에서 저장소 선택

방금 만든 GitHub 저장소 고르기

3 Deploy 버튼 한 번

몇 초 후 xxx.vercel.app 주소 생성됨

흔히 하는 실수

수정했는데 그대로 보임 → **재배포**됐는지, **새로고침**했는지 확인

이미지가 깨져서 보임 → **파일 경로** 대소문자·폴더 위치 확인

도메인 헛갈림 → **기본 제공 주소**로 충분, 따로 살 필요 없음

무료 한도 걱정 → **개인 포트폴리오 트래픽**으로는 거의 도달 안 함

* 배포(Deploy) — 만든 파일을 실제로 인터넷에 올리는 것 · 가져오기(Import) — GitHub 저장소를 Vercel로 연결하는 것

핵심 GitHub에 올려져 있으면 Vercel 연결은 클릭 몇 번이면 끝난다

AI 운영비용 — 한 달 커피값

이 가이드에서 쓰는 도구, 이 정도면 충분하다.

Claude Pro

\$20 /월

Claude Code 사용을 위한
필수 결제 — 이것만 있으면 끝

Vercel

무료

포트폴리오·웹사이트 배포는
무료 플랜으로 충분

GitHub

무료

백업·버전관리용 —
용량이 크지 않다면 충분

핵심 추가로 드는 돈은 없다 — Claude Pro \$20 결제 하나로 시작부터 배포까지 끝난다

CLAUDE CODE GUIDE · 2026

Thank You

질문은 언제든지 편하게 — 같이 만들어봅시다